MCP

Model Context Protocol

Why MCP?

What is the Model Context Protocol (MCP)

MCP is an open protocol that standardizes how your **LLM applications** connect to and work with your **tools & data sources**.

REST APIs

Standardize how **web applications** interact with the **backend**

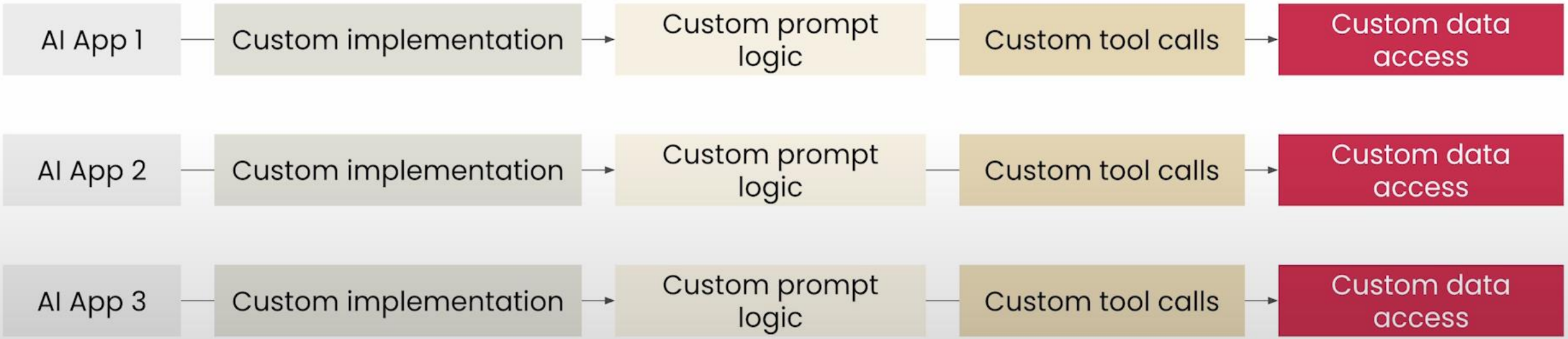
LSP

Standardizes how **IDEs** interact with **language-specific tools**

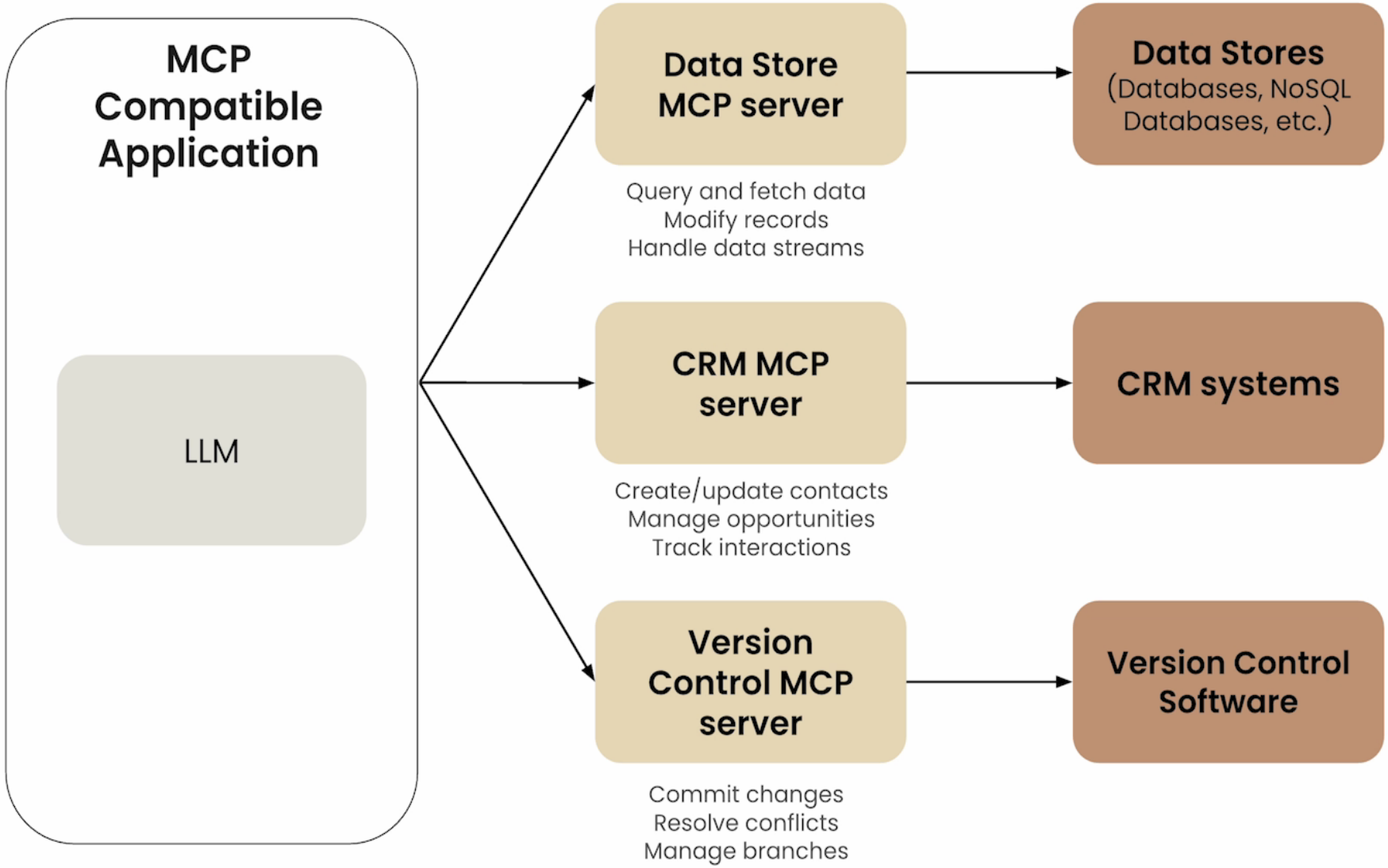
MCP

Standardizes how **AI applications** interact with **external systems**

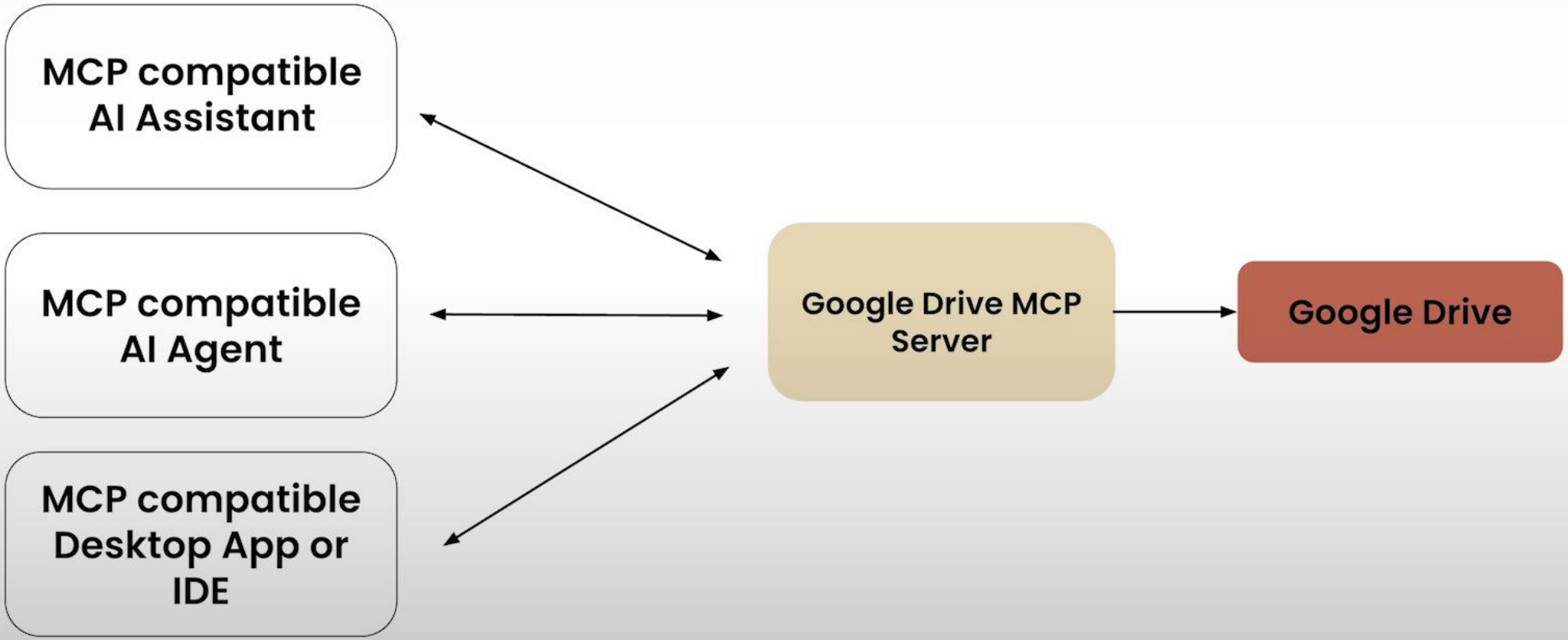
Without MCP: Fragmented AI Development



With MCP: Standardized AI Development



With MCP: MCP servers are reusable by various AI applications



With MCP: Standardized AI Development

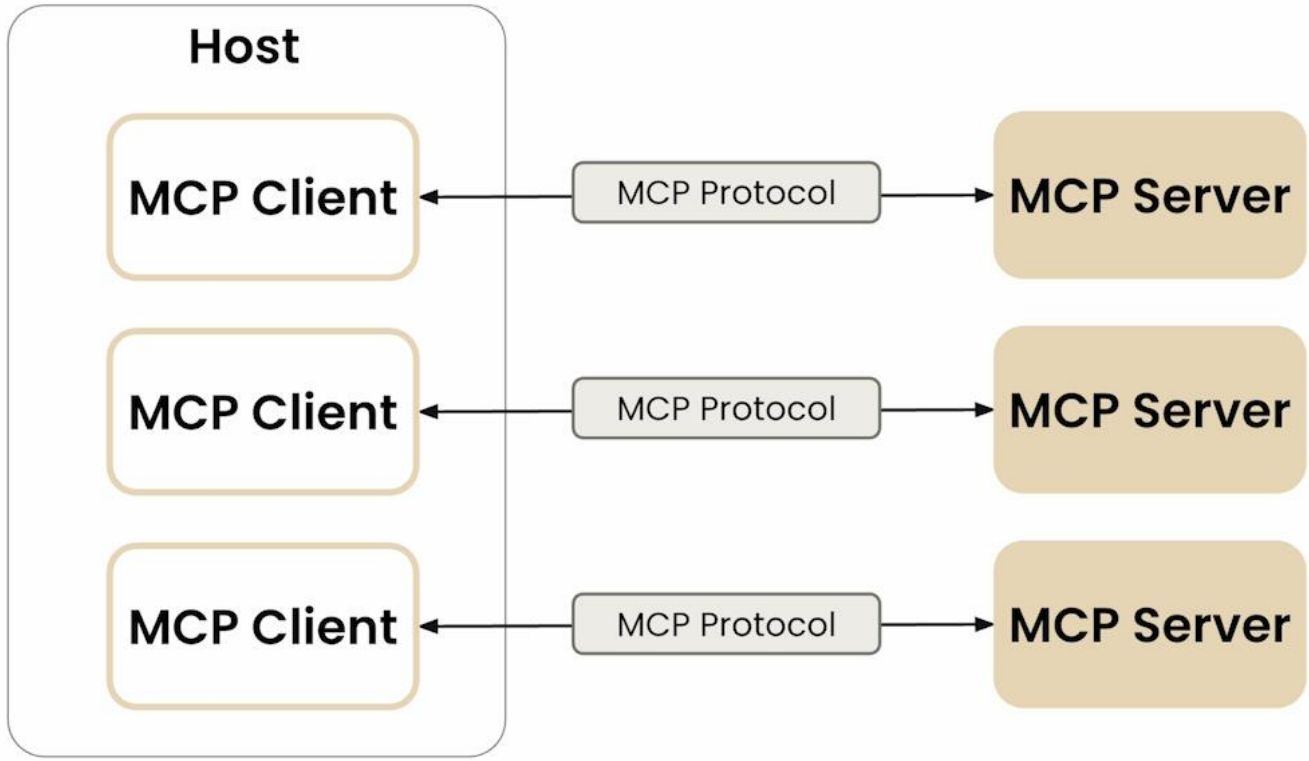
- **For AI application developers**
  Connect your app to any MCP server with 0 additional work
- **For tool or API developers**
  Build an MCP server once which can be adopted everywhere
- **For AI application users**
  AI applications have extensive capabilities
- **For enterprises**
  Clear separation of concerns between AI product teams

Common Questions

Who authors the MCP Server?	Anyone! Often the service provider itself will make their own MCP implementation. You can make a MCP server to wrap up access to some service.
How is using an MCP Server different from just calling a service's API directly?	MCP Servers provide tool schemas + functions. If you want to directly call an API directly, you'll be authoring those on your own.
Sounds like MCP Servers and tool use are the same thing.	MCP Servers provide tool schemas + functions already defined for you.

MCP Architecture
Build Rich Context AI-Apps

Client-Server Architecture

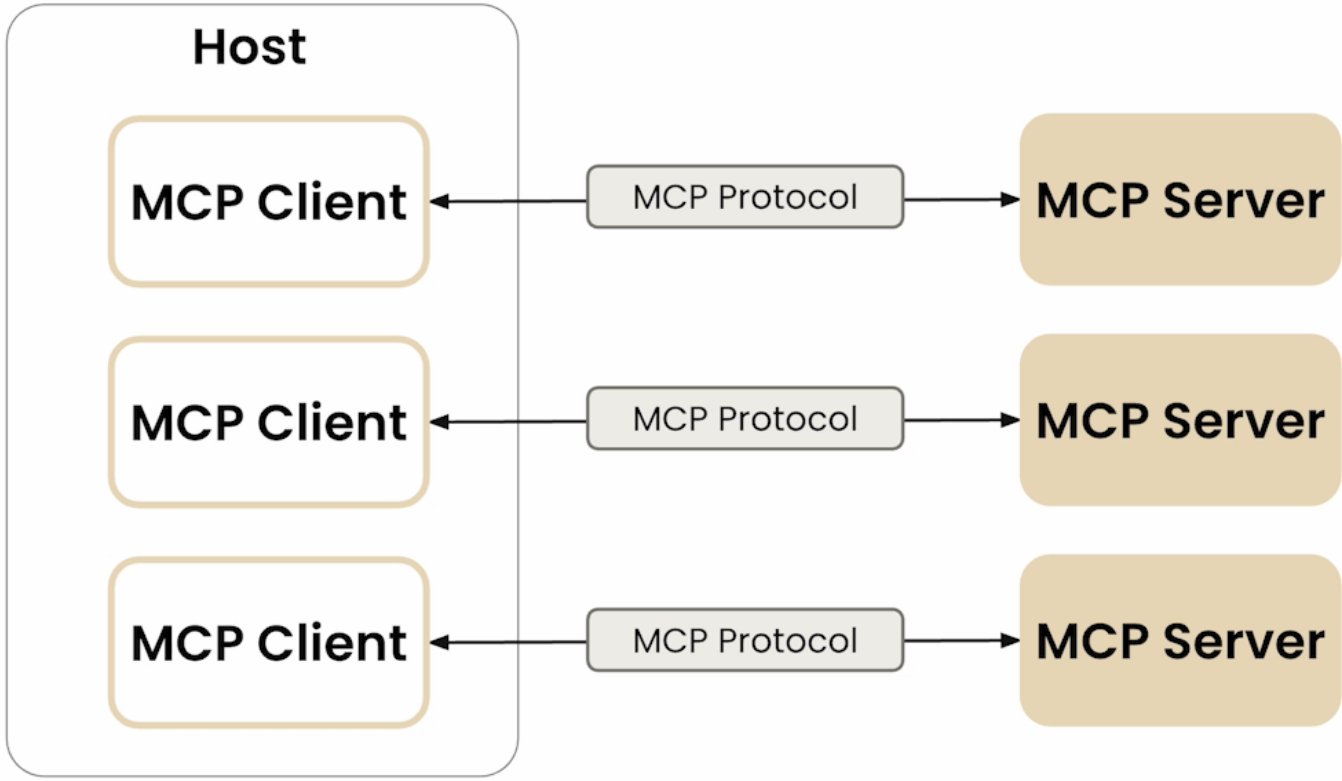


Client-Server Architecture

Host are LLM applications that want to access data through MCP (ex: Claude Desktop, IDEs, AI agents).

MCP Servers are lightweight programs that each expose specific capabilities through MCP.

MCP Clients maintain 1:1 connections with servers, inside the host application.



How does it work?

MCP Client

Invokes **Tools**
Queries for **Resources**
Interpolates **Prompts**

MCP Server

Exposes **Tools**
Exposes **Resources**
Exposes **Prompt Templates**

Tools

Functions and tools that can be invoked by the client

- Retrieve / search
- Send a message
- Update DB records

Resources

Read-only data or exposed by the server

- Files
- Database Records
- API Responses

Prompt Templates

Pre-defined templates for AI interactions

- Document Q&A
- Transcript Summary
- Output as JSON

Defining a Tool

- MCP provides SDK's for building servers and clients in a variety of languages
- You will be using the Python MCP SDK
- The Python SDK makes it very easy to declare tools

MCP Server

```python
@mcp.tool()
def add(a: int, b: int) -> int:
    """
    Args:
        a: First number to add
        b: Second number to add

    Returns:
        The sum of the two numbers
    """
    return a + b
```

Resources

- Allow the MCP Server to expose data to the client
- Similar to GET request handlers in a typical HTTP server
- Can return any type of data - strings, JSON, binary, etc.
  - You set the 'mime_type' to give the client a hint as to what data you are returning
- Two types: direct and templated

MCP Server

Direct

```
@mcp.resource(
    "docs://documents",
    mime_type="application/json"
)
def list_docs():
    # Return a list of document names
```

Templated

```
@mcp.resource(
    "docs://documents/{doc_id}",
    mime_type="text/plain"
)
def fetch_doc(doc_id: str):
    # Return the contents of a doc
```

Direct resources

```
> Can you please summarize the contents
of @
     deposition.md   Resource
     design.md       Resource
     financials.md   Resource
     outlook.md      Resource
     plan.md         Resource
     spec.md         Resource
```

...You need the MCP Server to give us a *list* of documents

Templated resources

```
Answer the users query:
<query>
What's in the @report.pdf file?
</query>

The user may have referenced a document.
Here is it's contents:
<document id="report.pdf">
...condition of a 20m condenser tower...
</document>
```

You need the MCP Server to give us the contents of a *single* document

Prompts

- Defines a set of User and Assistant messages that can be used by the client
- These prompts should be high quality, and well-tested.

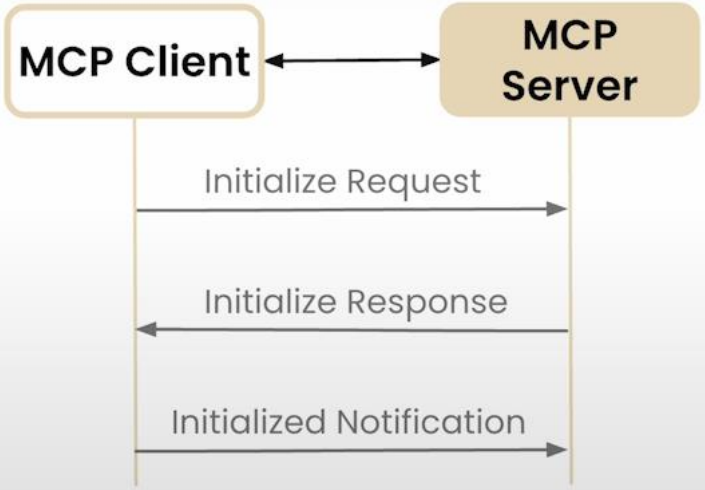
MCP Server

Prompt

```
@mcp.prompt(
    name="format",
    description="Rewrites the contents of
a document in Markdown format",
)
def format_document(
    doc_id: str,
) -> list[base.Message]:
    # Return a list of messages
```

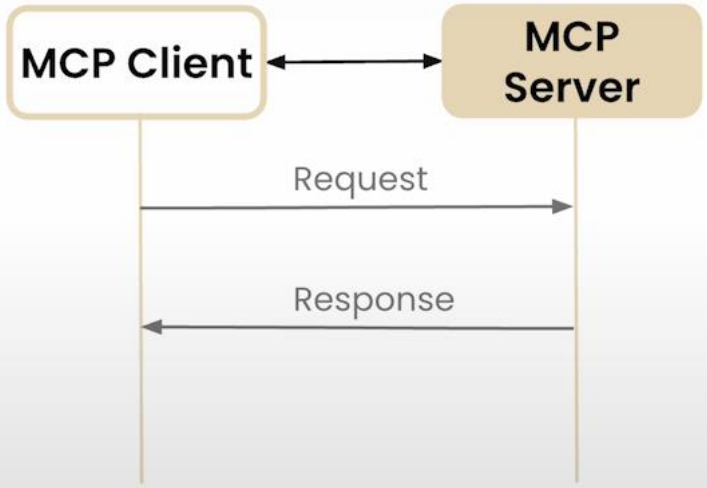
Communication Lifecycle

1. Initialization



Connection is established

2. Message Exchange



3. Termination

MCP Transports

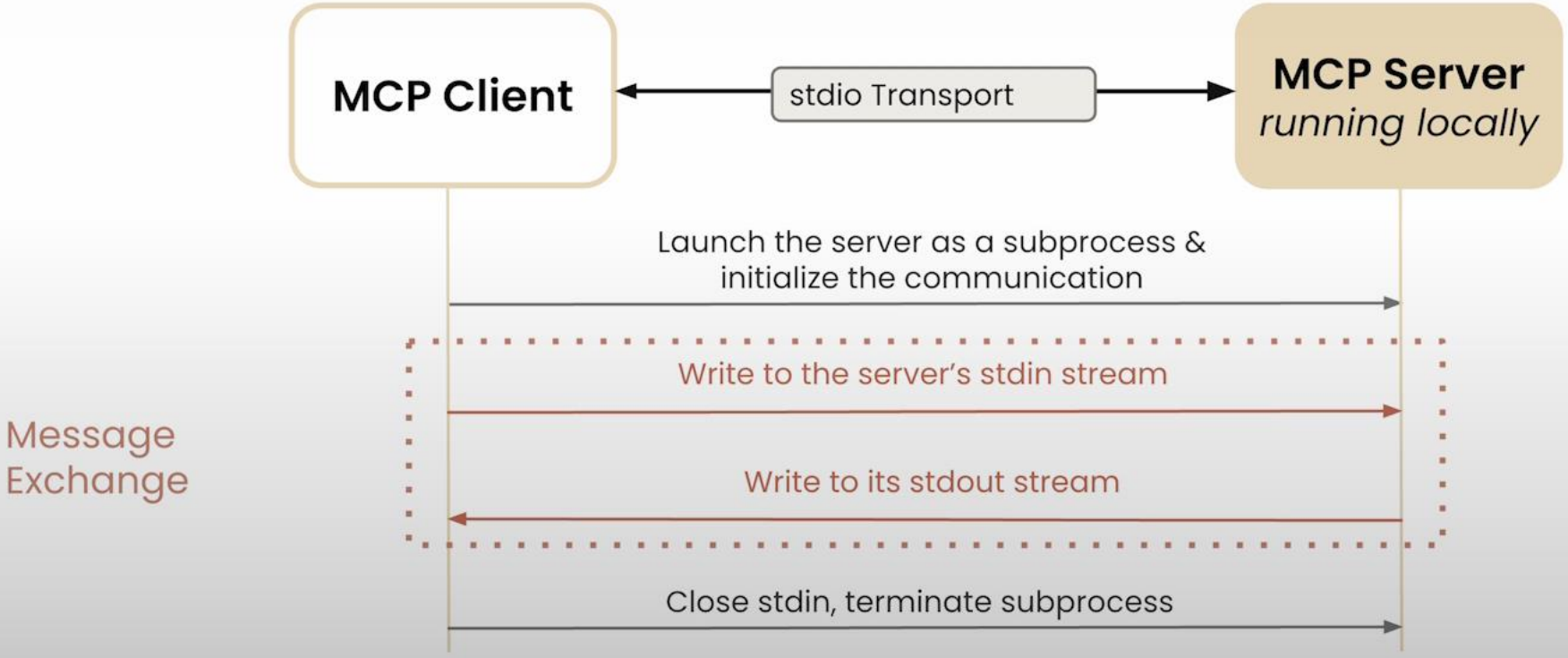
A transport handles the underlying mechanics of how messages are sent and received between the client and server.

1. For servers running locally: **stdio** (standard input output)

2. For remote servers:
   a. **HTTP+SSE (Server Sent Events)** (from protocol version 2024-11-05)
   b. **Streamable HTTP** (as of protocol version 2025-03-26)

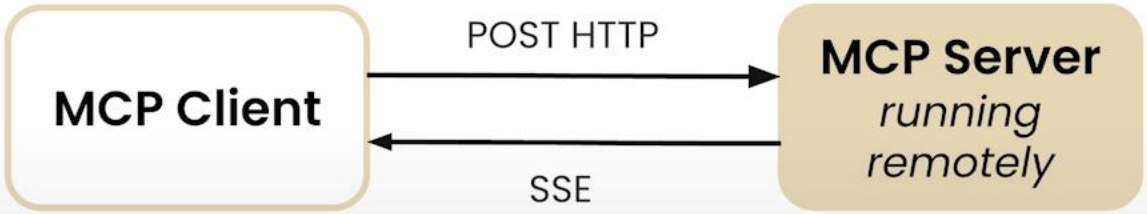
Standard IO (stdio) Transport

When running servers locally, stdio is most commonly used



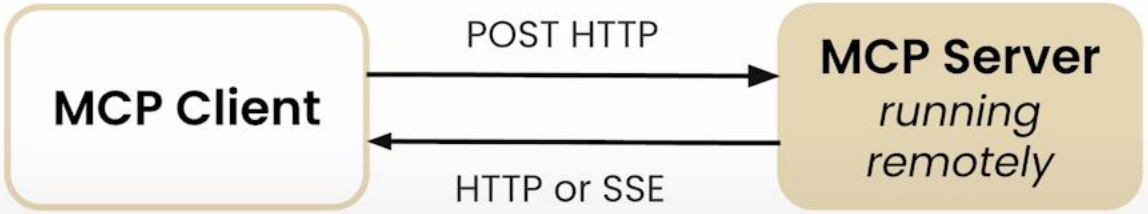
Transports for Remote Servers

HTTP + SSE
(from protocol version 2024-11-05)



Stateful Connection

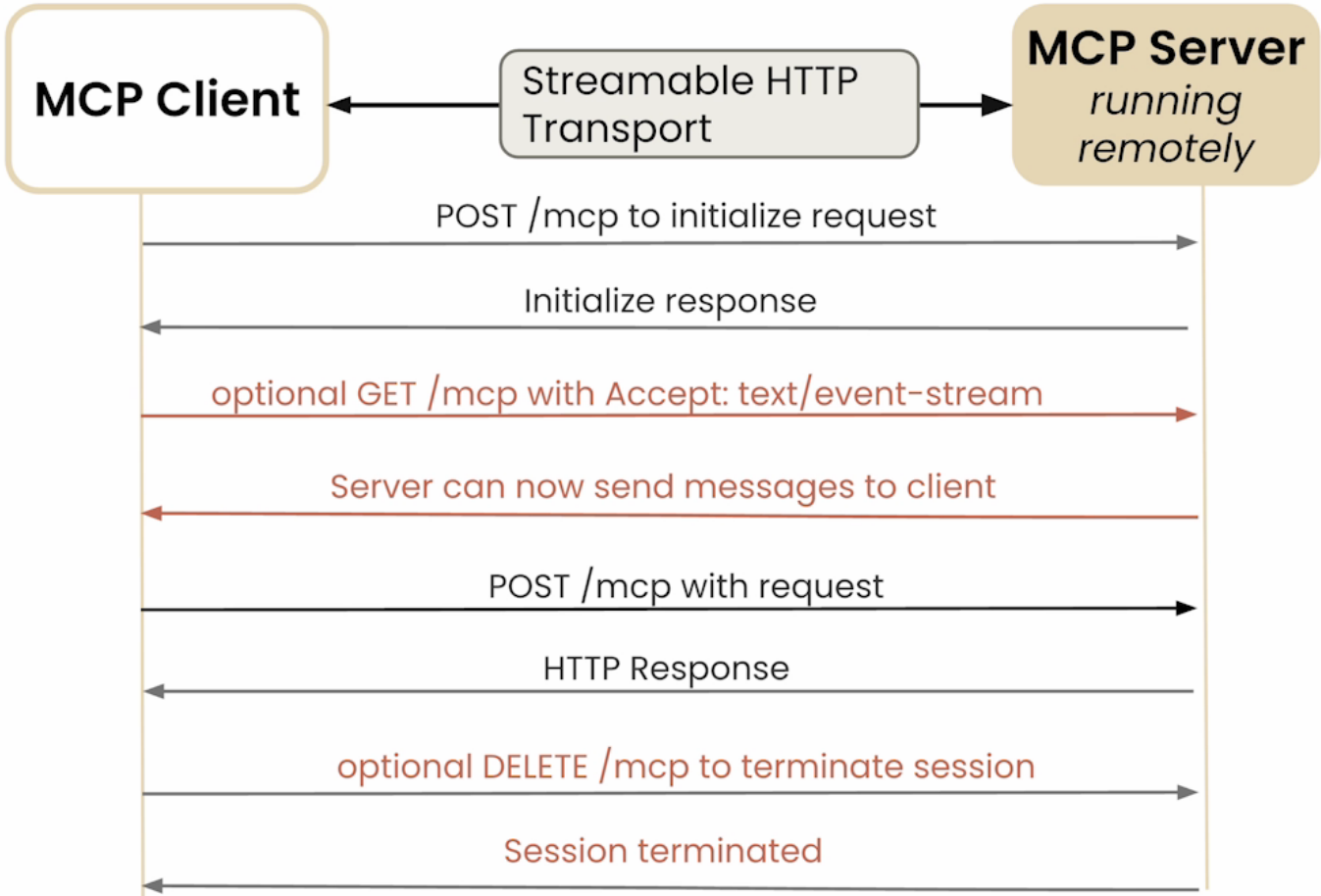
Streamable HTTP
(as of protocol version 2025-03-05)



Allow for Stateless or Stateful Connection

Streamable HTTP Transport

For remote MCP servers the Streamable HTTP transport is used. This supports stateless connections as well as stateful connections with the ability to opt into Server Sent Events



Demo

Demo SQLite DB accessible via MCP to AI assistant

Thank You